

Search Result Selector — Test Coverage Report

Sep 23, 2026 · prepared by @Someone

Why we did this

The selector will run on the business team's own test file during the meeting, and we have not seen that file. Rather than find out live whether an unusual row breaks the run, we deliberately fed the pipeline malformed and adversarial inputs beforehand and fixed anything that crashed or behaved unpredictably. The goal was not to improve match accuracy (that work is tracked separately) but to guarantee the pipeline always finishes and always produces output files, no matter what shape of data it is handed.

How we came up with the test cases

We worked outward from the two places bad data can enter the pipeline: the shape of the file as a whole, and the shape of a single product record inside it. For each place, we asked "what is the worst plausible mistake in a hand-edited or partially-exported JSON file" and turned each answer into a real test file, then ran it through the actual code rather than just reading the code and guessing.

1. **File-level shape errors** — the file isn't valid JSON at all, or it's valid JSON but the wrong shape (an array of strings instead of an object keyed by product).
2. **Per-product shape errors** — one product's value is not an object (a bare string, a list, or null), which is the kind of thing a broken export or a manual copy-paste mistake produces.
3. **Missing or wrong-type fields** — a product missing `product_title` entirely, or a field holding `null` or a number instead of text, or `search_results` holding a list instead of the expected object.
4. **Size and cost extremes** — a 10,000-character title, an uncapped result title, and a product with 25 search results when we normally expect far fewer, all of which could silently inflate token cost or blow past the model's limits.
5. **Content-level adversarial input** — text designed to make the model ignore its instructions (prompt injection), embedded both in a search-result snippet and, more unusually, in the product title/description themselves.

6. **Encoding and text edge cases** — unicode, emoji, right-to-left text, and control characters, since the business team's export could come from any system.

7. **Boundary conditions** — the exact character-count threshold where the pipeline decides a title is "too short to judge" (2 vs. 3 characters), and duplicate result content sent under different indices.

For each category we wrote the smallest input that would trigger it, ran it through `run.py`, and read the actual output and audit files rather than assuming the code would behave as intended.

Full test case catalog

23 cases across 5 files. All are kept as real, re-runnable sample files in `samples/adversarial/` (see below) so anyone can reproduce this table at any time with `python run.py samples/adversarial/<file>.json --backend heuristic`.

#	Test case	Category	What it simulates	Outcome
1	malformed_json_syntax.json	File-level shape	The file isn't valid JSON at all (a truncated copy/paste)	Fixed: was a raw Python traceback, now a one-line error and clean exit
2	malformed_top_level_array.json	File-level shape	File is a JSON array, not an object keyed by product	Fixed: was a raw traceback, now a one-line error and clean exit
3	malformed_product_value.json	Per-product shape	One product's value is a bare string, not an object	Fixed — most severe bug found. Was crashing the entire run and discarding every other product's results; now isolated to that one product
4	mixed_good_and_bad_products.json	Per-product shape	A batch mixing normal products with string/list/null/number product values	Fixed: previously wrote correct output files but then crashed while printing metrics; now completes cleanly end to end
5	empty_all	Per-product shape	Product with no title, description, or results	Pass — already abstained correctly
6	null_fields	Missing/wrong-type fields	title, description, and results all JSON <code>null</code>	Hardened — now explicitly guarded rather than relying on incidental behavior
7	missing_keys	Missing/wrong-type fields	Only <code>product_title</code> present, other fields absent	Pass

#	Test case	Category	What it simulates	Outcome
8	results_as_list	Missing/wrong-type fields	<code>search_results</code> is a list instead of an object	Fixed: previously could crash deep in result formatting; now coerced to empty safely
9	nonnumeric_indices	Missing/wrong-type fields	Result keys are letters ("a", "b") instead of numbers	Fixed: previously crashed on sorting; now skipped and reported in the audit
10	null_result_value	Missing/wrong-type fields	One search result's value is <code>null</code>	Fixed: crashed the offline/heuristic backend; now treated as empty
11	garbage_result_text	Missing/wrong-type fields	Result text has no recognizable Title/Link/Snippet structure	Pass
12	huge_title	Size / cost extremes	10,000+ character product title	Hardened — now capped at 300 characters so it can't inflate prompt size or cost
13	huge_result_title	Size / cost extremes	5,000 character Title and Snippet inside one search result	Hardened — result title now capped at 200 characters (snippet was already capped)
14	too_many_results	Size / cost extremes	25 search results for one product (normal cap is 15)	Hardened — the extra results were being silently dropped; now recorded in the audit as <code>results_dropped</code>
15	injection_in_upload	Adversarial content	"Ignore all previous instructions..." written into the product title/description itself	Hardened — now flagged in the audit (<code>upload_flag</code>) for visibility; the system prompt's untrusted-data instruction was already the actual defense
16	injection_evasion	Adversarial content	A differently-worded injection attempt designed to dodge our keyword filter	Accepted risk — documented below, not fixed with more keyword matching
17	unicode_stress	Encoding / text edge cases	Arabic, emoji, Japanese, and combining accent marks in title/description/results	Pass
18	control_chars	Encoding / text edge cases	Zero-width spaces, null bytes, and other control characters in text fields	Pass
19	title_len_2	Boundary conditions	Title exactly 2 characters (below the "too short to judge" cutoff)	Pass — correctly abstains
20	title_len_3	Boundary conditions	Title exactly 3 characters (at the cutoff)	Pass — correctly proceeds
21	exact_dup_content	Boundary conditions	Two results with identical title/link/snippet under different indices	Pass — deduplicated as designed

#	Test case	Category	What it simulates	Outcome
22	whitespace_only	Boundary conditions	Title and description are whitespace-only strings	Pass — correctly abstains
23	missing_links	Boundary conditions	Multiple results missing a Link field (checked for a dedup false-positive)	Pass — ruled out a suspected bug; results with no link are never falsely treated as duplicates of each other

Bugs found and fixed

One critical bug. If a single product in the input file was malformed — for example its value was a plain string instead of a structured record — the whole run used to crash partway through, after already doing the work, and no output file was written for *any* product, including the ones that succeeded. In a live meeting, one bad row anywhere in the business team’s file would have meant walking away with nothing. This is now isolated: that one product is recorded as invalid with its original value preserved, and every other product’s results are written normally.

Five lower-severity gaps, all now handled the same fail-closed way — flagged and skipped rather than trusted, never allowed to crash the run:

- Results beyond the normal 15-per-product limit were silently thrown away with no trace; they’re now recorded in the audit file.
- An unusually long product title or result title could inflate token cost with no limit; both are now capped.
- A result list of the wrong shape (e.g. an array instead of the expected structure) or with non-numeric keys could crash result parsing; both are now caught and reported instead.
- A prompt-injection attempt written into the product title or description itself was invisible in the audit trail (only injection inside a search-result snippet was being flagged); it’s now flagged for either location.

All of this is covered by 9 new automated tests added to `tests/test_pipeline.py` (24 tests total, all passing), plus the 23 end-to-end cases in the table above.

What we're accepting as a known risk

One finding was deliberately not "fixed" with more pattern-matching: a prompt-injection attempt worded differently from the phrases our keyword filter checks for (things like "ignore all previous instructions") can slip past that filter without being flagged in the audit trail.

This is expected and acceptable because the keyword filter was never the real defense — it only adds visibility for the obvious cases. The actual defense is structural: the system prompt explicitly tells the model that everything from the product upload and the web search results is untrusted data to classify, never instructions to follow, and the model's output is constrained to a strict schema (a fixed set of verdict labels) so there is no free-text field for a hijacked response to escape into. Trying to close this gap by adding more keywords is a losing game against a determined adversary; the schema constraint is what actually holds regardless of wording.

Where this lives and how to re-run it

- `samples/adversarial/` — the 5 test files behind the table above, kept as permanent, re-runnable sample data alongside the existing `samples/train_50.json`.
- `tests/test_pipeline.py` — 24 automated offline tests (`python tests/test_pipeline.py`), including 9 added specifically for this hardening pass. These run against a scripted fake model, so they need no API key and no cost.
- `README.md` — has a summary of this work under "Input-robustness hardening."

To reproduce any row in the table above:

```
python run.py samples/adversarial/product_level_edge_cases.json --backend heuristic
python run.py samples/adversarial/malformed_product_value.json --backend heuristic
python run.py samples/adversarial/mixed_good_and_bad_products.json --backend heuristic
```

`--backend heuristic` runs the offline lexical baseline (no API key, no cost) — all the robustness fixes above live in the shared input-handling code, so they apply identically whether the run uses the heuristic backend or the live OpenAI model.